PROYECTO STEAM 2000

Efecto 2000!

Lenguajes de Marca
Desarrollo de Aplicaciones Multiplataforma (DAM)
1º Año 2º Trimestre

Juan José Rivera
Juan Sebastián Segura
Jiandong Yao (Alejandro)

Marzo 2026

Juan José Rivera
Juan Sebastián Segura
Jiandong Yao (Alejandro)

Contenido

El proyecto utiliza navegación estática entre páginas HTML independientes. Cada sección (tienda, inventario, login, registro, detalle) es un archivo HTML separado con su propio script de JavaScript específico.

La barra de navegación es idéntica en todas las páginas y contiene enlaces directos con href a cada archivo HTML. Para la página de detalle de un juego, se pasa el identificador como parámetro en la URL (por ejemplo detalle.html?id=3), y el script detalle.js lee ese parámetro con URLSearchParams para buscar y mostrar la información del juego correspondiente.

Los datos compartidos entre páginas (tema visual, idioma, usuario logueado y juegos del inventario) se mantienen mediante localStorage, lo que permite que la información persista al navegar de una página a otra.

Juan José Rivera
Juan Sebastián Segura
Jiandong Yao (Alejandro)

1. Introducción

Este documento describe el desarrollo del proyecto Steam 2000 para la asignatura de Lenguajes de Marca. El objetivo del proyecto es crear una página web sobre un tema libre que muestre datos almacenados en archivos JSON y YAML, utilice Bootstrap para el diseño y permita interactividad mediante JavaScript.

El tema elegido es una recreación de la tienda de videojuegos Steam con la estética visual de principios de los años 2000, cuando la plataforma fue lanzada por Valve. La web incluye una tienda con catálogo de juegos, un sistema de inventario personal, y pantallas de inicio de sesión y registro de usuario.

2. Estructura del Proyecto

El proyecto está organizado en múltiples páginas HTML estáticas que comparten la misma barra de navegación, pie de página y archivos de estilo. La navegación entre secciones se realiza mediante enlaces normales entre páginas.

2.1 Archivos del proyecto

La estructura de carpetas y archivos es la siguiente:

Archivo / Carpeta	Descripción
index.html	Página principal: tienda con el catálogo de juegos
detalle.html	Ficha con la información completa de un juego
inventario.html	Inventario personal del usuario (juegos añadidos)
login.html	Formulario de inicio de sesión
registro.html	Formulario de registro de nueva cuenta
data/games.json	Datos principales: catálogo de 12 juegos con sus propiedades
data/config.yaml	Configuración de la aplicación: tema, idioma, paginación, categorías
css/styles.css	Estilos personalizados con temática retro Steam 2002
js/dataLoader.js	Funciones para cargar los archivos JSON y YAML con fetch()
js/traducciones.js	Textos en español e inglés y funciones de cambio de idioma
js/common.js	Funciones compartidas: tema, estrellas, placeholders, inventario
js/tienda.js	Lógica de la página de tienda: filtros, vistas, paginación
js/detalle.js	Lee el id de la URL y muestra la ficha del juego
js/inventario.js	Muestra y gestiona los juegos del inventario en localStorage
js/login.js	Validación del formulario de login y guardado en localStorage
js/registro.js	Validación del formulario de registro

2.2 Páginas HTML

Cada página HTML sigue la misma plantilla básica: incluye Bootstrap 5 mediante CDN, la librería js-yaml para leer archivos YAML, y nuestro archivo de estilos personalizado. La barra de navegación y el pie de página se copian idénticamente en todas las páginas. Los scripts de JavaScript se cargan al final del body en un orden específico para que las dependencias entre archivos funcionen correctamente.

3. Estructura de Datos

3.1 Archivo JSON (games.json)

El archivo games.json contiene un array con 12 objetos, cada uno representando un juego de la época del año 2000. Cada objeto tiene las siguientes propiedades:

- id: número identificador único del juego
- title: nombre del juego (por ejemplo Half-Life, Counter-Strike, Diablo II)
- genre: género del juego (fps, rpg, estrategia, simulacion, aventura)
- year: año de lanzamiento (entre 1998 y 2000)
- price: precio en dólares
- developer: nombre del estudio que desarrolló el juego
- description: descripción corta del juego en una o dos frases
- rating: puntuación del 1 al 5
- platform: array con las plataformas disponibles (PC, Mac)
- onSale: booleano que indica si el juego está en oferta

Este archivo se carga desde JavaScript usando fetch() y response.json() en la función cargarJuegos() del archivo dataLoader.js.

3.2 Archivo YAML (config.yaml)

El archivo config.yaml almacena la configuración de la aplicación. Se divide en varias secciones:

- app: información básica como el nombre, versión e idioma por defecto
- display: opciones de visualización como el número de juegos por página (items_per_page: 6), el tema visual (dark/light), si mostrar imágenes y el modo de vista (cards/list)
- animations: tipo de animación de transición y su duración
- categories: lista de categorías para filtrar juegos, cada una con su etiqueta en español e inglés

Este archivo se carga como texto plano con fetch() y se convierte a un objeto JavaScript usando la librería js-yaml mediante jsyaml.load(). La función cargarConfiguracion() de dataLoader.js se encarga de este proceso.

4. Diseño con Bootstrap

4.1 Uso de Bootstrap 5

Todo el proyecto utiliza Bootstrap 5 cargado mediante CDN. Se emplean los siguientes componentes de Bootstrap:

- Navbar: barra de navegación responsiva con menú hamburguesa para móviles
- Cards: tarjetas para mostrar cada juego con su imagen, título, estrellas, género y precio
- Tables: tablas para la información detallada del juego y el inventario
- Forms: formularios con validación usando las clases is-invalid e invalid-feedback
- Badges: etiquetas para las plataformas del juego y el indicador de oferta
- Grid system: sistema de columnas responsivo (col-md-4, col-lg-3) para organizar las tarjetas
- List group: vista alternativa de lista para los juegos
- Alerts: mensajes de éxito y error en formularios e inventario

4.2 Estética retro Steam 2002

El archivo styles.css sobreescribe los estilos de Bootstrap para darle a la web el aspecto visual de Steam en sus primeros años. Los elementos más característicos son:

- Paleta de colores verde oliva: se usan colores como #4a5942, #5c7e10 y #98bf0c, que eran los tonos característicos de Steam en 2002
- Tipografía Verdana a 12px: la fuente que usaban la mayoría de webs de esa época
- Sin bordes redondeados: todo tiene esquinas cuadradas (border-radius: 0) para conseguir el look plano de la época
- Fondo gris oscuro: el fondo principal es un gris oscuro (#3b3b3b) con tarjetas en gris aún más oscuro (#2e2e2e)
- Variables CSS: se definen 5 variables de color para cada tema (oscuro y claro), lo que permite cambiar el aspecto visual fácilmente

5. Interactividad

5.1 Filtrado por categoría

La página de tienda incluye un select (desplegable) que permite filtrar los juegos por género. Las opciones del desplegable se cargan dinámicamente desde las categorías definidas en el archivo YAML. Al seleccionar una categoría, JavaScript filtra el array de juegos y vuelve a generar el HTML mostrando solo los que coinciden.

5.2 Vista cards y lista

El usuario puede cambiar entre dos modos de visualización: tarjetas (cards) y lista. Dos botones en la barra de filtros permiten alternar entre ambas vistas. El modo de vista inicial se lee de la configuración YAML (campo view_mode).

5.3 Paginación

La cantidad de juegos que se muestran a la vez está limitada por el valor items_per_page del archivo YAML (por defecto 6). Si hay más juegos disponibles, aparece un botón de cargar más que añade el siguiente lote de juegos al contenedor.

5.4 Configuración desde YAML

La aplicación aplica varias configuraciones leídas del archivo YAML:

- Tema visual (oscuro o claro): se aplica añadiendo una clase CSS al body
- Número de elementos por página: controla cuántos juegos se muestran a la vez
- Modo de vista: determina si se muestran tarjetas o lista al cargar la página
- Mostrar imágenes: controla si los placeholders de imagen aparecen o no
- Idioma: el idioma por defecto de la interfaz

5.5 Tema oscuro y claro

Un switch en la barra de navegación permite cambiar entre el tema oscuro (por defecto, estética Steam original) y un tema claro (estilo Windows 2000). El tema elegido se guarda en localStorage para que se mantenga al navegar entre páginas.

5.6 Cambio de idioma

Un selector de idioma en la barra de navegación permite cambiar entre español e inglés. Al cambiar, el idioma se guarda en localStorage y la página se recarga para que los textos de la interfaz (navbar, footer, botones principales) se actualicen. Las etiquetas de las categorías del filtro también cambian, usando los campos label_es y label_en definidos en el YAML.

5.7 Inventario con localStorage

El sistema de inventario utiliza localStorage del navegador para almacenar los identificadores de los juegos que el usuario añade. Desde la ficha de detalle de un juego se puede añadir al inventario, y desde la página de inventario se pueden ver y eliminar los juegos guardados. Los datos persisten aunque se cierre el navegador.

5.8 Login y registro

Los formularios de inicio de sesión y registro incluyen validación básica con JavaScript: campos vacíos, formato de email y coincidencia de contraseñas. Al iniciar sesión, se guarda el usuario en localStorage y la barra de navegación muestra su nombre en lugar del enlace

de login. No se conecta a ningún servidor real; es una simulación.

7. Memoria

1 Selección del tema

El proyecto parte del enunciado «Efecto 2000» de la asignatura Lenguajes de Marca, que pide desarrollar una web dinámica sobre un tema libre. Los datos deben almacenarse en archivos JSON y YAML, la interfaz debe usar Bootstrap, y la aplicación debe permitir navegación entre secciones sin recargar la página.

El equipo eligió como tema una recreación de la tienda de videojuegos Steam con la estética visual de principios de los años 2000, cuando Valve lanzó la plataforma. Esta elección permitía cubrir todos los requisitos del enunciado: un catálogo de juegos como datos principales en JSON, configuraciones de la aplicación en YAML, y múltiples secciones (tienda, inventario, login, registro) que requieren navegación e interactividad.

2 División de roles

Siguiendo la recomendación del enunciado, el trabajo se distribuyó en tres roles:

Rol	Responsabilidades
Gestor de datos	Diseño del archivo games.json con los 12 juegos y sus propiedades. Diseño del archivo config.yaml con la configuración de la aplicación. Implementación de dataLoader.js: funciones cargarJuegos() y cargarConfiguracion() usando fetch() y js-yaml.
Diseñador de interfaz	Estructura HTML de las cinco páginas (index, detalle, inventario, login, registro). Estilos CSS con la estética retro de Steam 2002: paleta verde oliva, Verdana 12px, sin border-radius. Integración de Bootstrap 5: navbar, cards, tablas, formularios, grid responsivo.
Desarrollador de interactividad	Filtrado de juegos por categoría y cambio entre vista tarjetas/lista. Paginación con botón «Cargar más». Sistema de inventario con localStorage. Validación de formularios de login y registro. Cambio de tema oscuro/claro y selector de idioma. Navegación estática entre páginas con paso de parámetros por URL.

3 Planificación semanal

Se estableció un calendario de trabajo de una semana (lunes 16 a domingo 22 de marzo) dividido en fases incrementales, de modo que cada fase construyera sobre la anterior sin romper lo ya funcional.

4. Desarrollo día a día

4.1 Lunes 16 — Estructura y datos

Se creó la estructura de carpetas del proyecto y se generaron los dos archivos de datos:

- games.json: un array de 12 objetos con propiedades id, title, genre, year, price, developer, description, rating, platform y onSale. Los juegos seleccionados corresponden a títulos reales de la época 1998–2000 (Half-Life, Counter-Strike, Diablo II, Age of Empires II, etc.).
- config.yaml: organizado en secciones app (nombre, versión, idioma), display (items_per_page, theme, show_images, view_mode), animations (tipo y duración) y categories (lista de géneros con etiquetas bilingües label_es y label_en).

También se implementó dataLoader.js con dos funciones asíncronas: cargarJuegos() que usa fetch() y response.json(), y cargarConfiguracion() que usa fetch(), response.text() y jsyaml.load() de la librería js-yaml cargada por CDN. Ambas funciones incluyen bloques try/catch con valores por defecto en caso de error.

Punto técnico relevante: YAML se carga como texto plano porque los navegadores no tienen un parser YAML nativo, a diferencia de JSON que se convierte directamente con response.json(). La librería js-yaml actúa como puente entre el formato YAML y los objetos JavaScript.

4.2 Martes 17 — Diseño inicial SPA

La primera versión del proyecto se diseñó como una aplicación de página única (SPA) con navegación por hash (#tienda, #inventario, #login, #registro). Se creó un único index.html con un router.js que escuchaba el evento hashchange y ejecutaba funciones de renderizado dinámico. Un archivo components.js contenía todas las funciones de generación de HTML (renderizarTienda, renderizarInventario, renderizarLogin, renderizarRegistro) y un app.js inicializaba todo.

Se implementó también un sistema de traducciones completo en traducciones.js con más de 80 claves en español e inglés, cubriendo desde los textos de la navbar hasta cada label y placeholder de los formularios.

El CSS usaba la paleta moderna de Steam (azul oscuro #1b2838, azul claro #66c0f4) con efectos de hover con transformaciones y sombras.

4.3 Miércoles 18 — Conversión a páginas estáticas

Se tomó la decisión de convertir el proyecto de SPA a múltiples páginas estáticas. El motivo fue que el enrutamiento dinámico por hash, con funciones que generaban todo el HTML desde JavaScript, excede el nivel de complejidad esperado para la asignatura. Las redirecciones estáticas con enlaces <a href> entre archivos HTML son más acordes a un curso introductorio.

Los cambios realizados fueron:

- Se eliminaron router.js, components.js y app.js.

- Se crearon cinco páginas HTML independientes: index.html (tienda), detalle.html, inventario.html, login.html y registro.html.
- Se extrajeron las funciones compartidas a common.js (tema, estrellas, placeholders, inventario).
- Se creó un script específico por página: tienda.js, detalle.js, inventario.js, login.js, registro.js.
- Cada página copia la misma navbar y footer, con atributos data-seccion en los enlaces para que traducciones.js los identifique.

Punto técnico relevante: al pasar de SPA a páginas estáticas, el estado compartido (tema, idioma, usuario, inventario) ya no puede vivir en variables JavaScript globales porque cada navegación recarga la página. La solución fue usar localStorage para persistir estos datos entre páginas. Para la página de detalle, se pasa el identificador del juego como parámetro en la URL (?id=3) y se lee con URLSearchParams.

4.4 Jueves 19 — Estética retro Steam 2002

Se rediseñó completamente el archivo styles.css para aplicar la estética original de Steam en 2002-2003. Se investigaron capturas de pantalla de la época a través del Web Design Museum y foros de la comunidad Steam.

Los elementos característicos implementados fueron:

- Paleta de colores verde oliva (#4a5942, #5c7e10, #98bf0c) en lugar del azul moderno de Steam.
- Tipografía Verdana a 12px, que era el estándar de las webs de esa época.
- Eliminación total de bordes redondeados con * { border-radius: 0 !important }, consiguiendo el aspecto cuadrado característico.
- Degradado verde en la navbar simulando la barra de navegación original de Steam.
- Variables CSS para dos temas: oscuro (fondo gris #3b3b3b con tarjetas #2e2e2e) y claro (fondo beige #d4d0c8 estilo Windows 2000).

Punto técnico relevante: la regla * { border-radius: 0 !important } es una solución global que sobreescribe todos los bordes redondeados de Bootstrap. El uso de !important es necesario aquí porque Bootstrap aplica border-radius inline en algunos componentes. Las variables CSS permiten cambiar el tema visual con solo añadir una clase al body (tema-oscuro o tema-claro), sin modificar ningún otro archivo.

4.5 Viernes 20 — Simplificación del código

Se realizó una revisión para reducir la complejidad del código al nivel esperado de un estudiante de primer año. Los cambios principales fueron:

- Traducciones: se redujeron de más de 80 claves a unas 14 esenciales. Los textos de formularios (labels, placeholders, mensajes de error) se dejaron directamente en español en el HTML en lugar de traducirlos dinámicamente.

- Placeholder de imágenes: se sustituyó un sistema elaborado con gradientes por género y tres tamaños por un simple div gris con el título del juego en blanco.
- Detalle del juego: se reemplazó un modal dinámico de Bootstrap (creado con insertAdjacentHTML y la API de Bootstrap Modal) por una página estática detalle.html que lee el parámetro ?id de la URL.
- CSS: se eliminaron efectos avanzados (bisel Windows 2000, patrón de píxeles en base64, 12 variables por tema) y se redujeron a 5 variables por tema con bordes sencillos.
- Formularios: se eliminaron las funciones de traducción por página (traducirTextosLogin, traducirTextosRegistro). Los scripts de login y registro solo inicializan, validan y redirigen.

También se sustituyeron los métodos modernos de arrays (forEach, find, filter, some) por bucles for clásicos y se reemplazó let/const por var en los archivos simplificados, dando un aspecto más coherente con un nivel principiante.

4.6 Sábado 21 — Documentación

Se redactó el informe técnico del proyecto en formato Word, cubriendo la estructura del proyecto, los datos JSON y YAML, el diseño con Bootstrap, la interactividad, la navegación y las tecnologías utilizadas. Se generó también un diagrama de mapa del sitio mostrando las cinco páginas como miniaturas con sus conexiones de navegación.

4.7 Domingo 22 — Revisión final

Se realizó la revisión final del proyecto verificando que todas las funcionalidades requeridas por el enunciado están operativas.